



5.2. Tower of Hanoi

"To iterate is human, to recurse is divine."

Main Section:

5. Recursion

Contents:

[1. Problem Definition](#)

[2. The Recursive Strategy](#)

[3. The Core Idea](#)

[Summary](#)

[References](#)

People often think Recursion is difficult because it feels unnatural at first. A function calling itself? Solving a problem by repeatedly breaking it into smaller versions of itself? It sounds strange, almost paradoxical.

But then there is the Tower of Hanoi. A puzzle so simple that even children can understand its rules, yet so elegant that it became one of the most iconic demonstrations of Recursion in Computer Science. And honestly, I think that is what makes it beautiful.

The Tower of Hanoi is not just a puzzle about moving disks between pegs. It is a lesson about decomposition. About reducing impossibly large problems into smaller, manageable ones. The kind of thinking that quietly appears everywhere in Computer Science: Searching; Divide and Conquer; Backtracking; Dynamic Programming. Even broadly Artificial Intelligence.

The first time I studied the Tower of Hanoi, I realized something important: Recursion is not about repeatedly calling functions. It is about trust. You trust

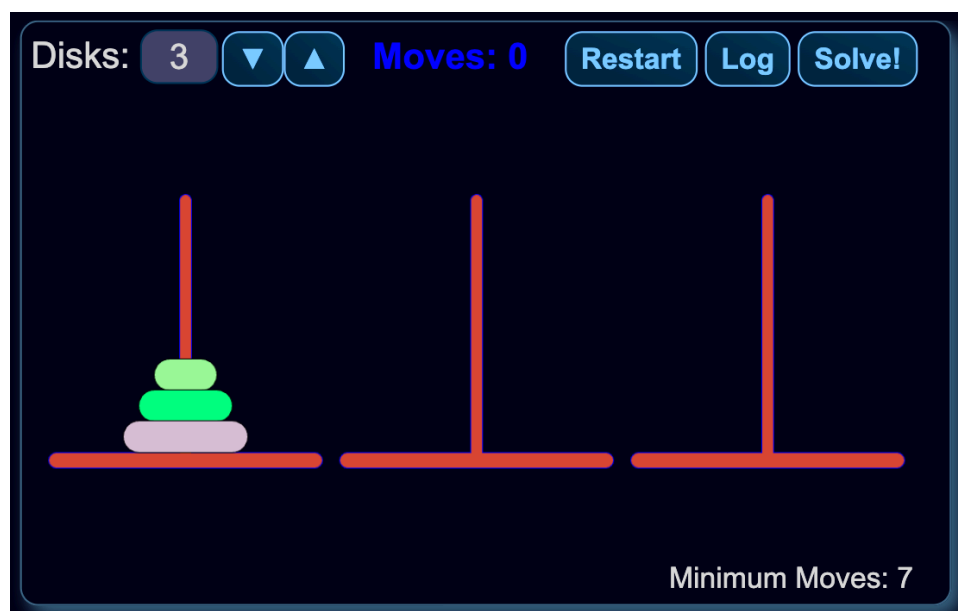
that a smaller version of the problem can solve itself. And once you trust that, the larger solution begins to unfold naturally.

So in this section, we will study one of the purest recursive problems ever created. If you truly understand the Tower of Hanoi, then you may finally understand the philosophy behind Recursion itself.

1. Problem Definition

The Tower of Hanoi is a mathematical puzzle consisting of three pegs and several disks of different sizes.

Reference the image below:



At the beginning, all disks are stacked on the first peg in descending order of size.

- The largest disk is placed at the bottom.
- The smallest disk is placed at the top

The objective is simple: ***Move the entire stack from the first peg to the third peg while preserving the original order of the disks.***

Of course, there is a catch. The puzzle follows three strict rules:

- Only one disk can be moved at a time

- **A disk can only be taken from the top of a stack**
- **A larger disk can never be placed on top of a smaller disk**

To avoid confusion, we usually define the three pegs as:

- **Source Peg (Peg A)**
- **Auxiliary Peg (Peg B)**
- **Destination Peg (Peg C)**

Now here is where things become interesting. If there is only **1 disk**, then the solution is trivial: Move the disk directly from Peg A to Peg C. Only **1 move** is needed.

If there are **2 disks**, then:

1. Move the smaller disk to Peg B
2. Move the larger disk to Peg C
3. Move the smaller disk from Peg B to Peg C

Total: **3 moves**.

For **3 disks**, the number suddenly becomes **7 moves**. And if you continue observing the pattern carefully, something remarkable appears:

$$m = 2^n - 1$$

Where:

- n = number of disks
- m = minimum number of moves required

This means the puzzle grows exponentially. Every additional disk nearly doubles the amount of work required.

And this is exactly why the Tower of Hanoi is such a powerful recursive problem. Because solving it directly becomes increasingly difficult, but solving a smaller version of it remains manageable.

The real objective of the recursive solution is therefore not to solve the entire puzzle immediately. It is to repeatedly reduce the problem until we finally arrive

at the simplest possible state: $n = 1$. The base case. One disk. One move. One final action: Move the disk from the source peg to the destination peg. Everything else in the Tower of Hanoi, is simply the repetition of this idea on larger scales.

2. The Recursive Strategy

Now comes the interesting part. We already know that:

- $n = 1$ requires 1 move
- $n = 2$ requires 3 moves
- $n = 3$ requires 7 moves

But the real question is: **How do we actually solve it recursively?** To understand this properly, let us focus on the most famous case: $n = 3$. Three disks. Seven moves. Three pegs.

At first glance, the puzzle looks complicated. But Recursion teaches us something extremely important:

Large problems become manageable once they are reduced into smaller versions of themselves.

And that is exactly how the Tower of Hanoi is solved. Instead of trying to move all three disks at once, we break the problem into three smaller tasks:

1. Move $(n - 1)$ disks from the source peg to the auxiliary peg
2. Move the largest disk directly to the destination peg
3. Move $(n - 1)$ disks from the auxiliary peg to the destination peg

For the Tower of Hanoi, we define the recursive function as:

```
TowerOfHanoi(n, source, destination, auxiliary)
```

Where:

- `n` → number of disks
- `source` → starting peg
- `destination` → target peg

- `auxiliary` → helper peg

The Base Case

As with every recursive problem, everything revolves around the base case. When: $n = 1$, the solution becomes trivial. There is only one disk remaining, so we simply move it directly from the source peg to the destination peg. No further recursion is needed.

Recursive Breakdown for ($n = 3$)

Initially, the function is called as:

```
TowerOfHanoi(3, A, C, B)
```

Meaning: ***Move 3 disks from Peg A to Peg C → Using Peg B as auxiliary***

Now the recursive process begins.

First Recursive Call

The first recursive strategy is:

```
TowerOfHanoi(n - 1, source, auxiliary, destination)
```

Meaning: Move the smaller ($n - 1$) disks away first. So the largest disk can eventually move freely

For ($n = 3$) :

```
TowerOfHanoi(2, A, B, C)
```

This means: ***Move 2 disks from A to B → Using C as auxiliary***

Still not the base case. So recursion continues again:

```
TowerOfHanoi(1, A, C, B)
```

Now we finally reach the base case. Thus: ***Disk 1 moves from A → C***

Recursion returns to the previous step.

- Then: Disk 2 moves from A → B
- And finally: Disk 1 moves from C → B

So the first recursive stage resolves into:

```
Disk 1: A → C
Disk 2: A → B
Disk 1: C → B
```

At this point: Peg B contains the two smaller disks. Peg A only contains the largest disk

Moving the Largest Disk

Now the recursive calls pause briefly. The largest disk can finally move directly:

```
Disk 3: A → C
```

This is the central move of the entire solution. Everything before this step existed only to free Disk 3. Everything after this step exists to rebuild the stack on top of it.

Second Recursive Call

Now we execute the second recursive strategy:

```
TowerOfHanoi( $n - 1$ , auxiliary, destination, source)
```

Meaning: Move the smaller disks from the auxiliary peg to the destination peg
→ Using the original source peg as helper

For ($n = 3$):

```
TowerOfHanoi( $2$ , B, C, A)
```

Again, recursion continues:

```
TowerOfHanoi( $1$ , B, A, C)
```

- Base case reached: **Disk 1 moves from B → A**
- Return to previous step: **Disk 2 moves from B → C**
- Final recursive return: **Disk 1 moves from A → C**

Thus, the second recursive stage becomes:

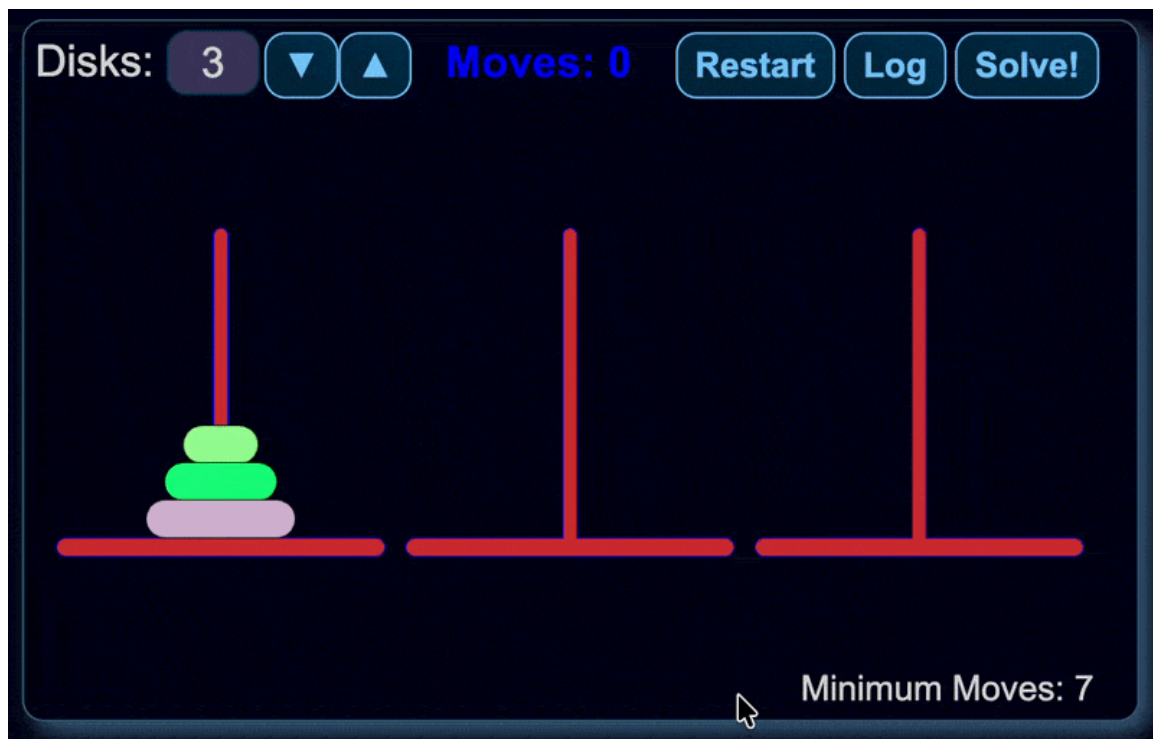
```
Disk 1: B → A  
Disk 2: B → C  
Disk 1: A → C
```

Final Sequence for ($n = 3$)

Combining all recursive stages together:

```
Disk 1: A → C  
Disk 2: A → B  
Disk 1: C → B  
Disk 3: A → C  
Disk 1: B → A  
Disk 2: B → C  
Disk 1: A → C
```

Below is the animation of solving Tower of Hanoi with $n = 3$ (3 disks)



Exactly 7 moves. And more importantly, every step emerged naturally from solving smaller versions of the same problem. That is the essence of Recursion.

Bringing the Idea into Code

What makes the Tower of Hanoi fascinating is that despite its complexity, the recursive implementation itself is surprisingly short.


```

# Tower of Hanoi
def tower_of_hanoi(num, source, destination, auxiliary):
    # Base Case
    if num == 1:
        print(f"Move Disk {num} from Peg {source} to Peg {d
estination}")
        return
    else:

        # Move smaller disks from source to auxiliary
        # using destination as helper
        tower_of_hanoi(num - 1, source, auxiliary, destinat
ion)

        # Move the largest disk directly
        print(f"Move Disk {num} from Peg {source} to Peg {d
estination}")

        # Move smaller disks from auxiliary to destination
        # using source as helper
        tower_of_hanoi(num - 1, auxiliary, destination, sou
rce)

# Peg A = Source
# Peg C = Destination
# Peg B = Auxiliary
tower_of_hanoi(3, "A", "C", "B")

```

Move Disk 1 from Peg A to Peg C
 Move Disk 2 from Peg A to Peg B
 Move Disk 1 from Peg C to Peg B
 Move Disk 3 from Peg A to Peg C
 Move Disk 1 from Peg B to Peg A
 Move Disk 2 from Peg B to Peg C
 Move Disk 1 from Peg A to Peg C

And this is one of the most important lessons in Recursion: The code itself is often small. The real challenge is understanding the flow of recursive thinking behind it.

3. The Core Idea

Now that we have seen the recursive calls themselves, let us finally talk about the real idea behind the Tower of Hanoi. Because honestly, the code is not the important part. The strategy is. If you truly understand the strategy, the recursive solution suddenly feels almost inevitable.

At its core, the Tower of Hanoi is built around one deceptively simple principle:

If you want to move a large disk, you must first move every smaller disk above it somewhere else.

That is the entire philosophy of the puzzle. And once you realize this, the recursive structure begins to emerge naturally.

The Hidden Objective

When beginners first see the Tower of Hanoi, they usually think:

“The goal is to move all the disks.”

But Recursion thinks differently. Recursion isolates the *real* problem. And the real problem is not moving all disks simultaneously. It is moving the **largest disk**.

Because the largest disk is trapped underneath every smaller one above it.

So the strategy quietly transforms:

Move all disks → Move the largest disk → Move the smaller disks out of the way

And suddenly, the impossible problem becomes manageable.

The Recursive Reduction

Suppose we have three disks. We want to move Disk 3 — the largest disk — from Peg A to Peg C. But we cannot do that immediately, because: **Disk 1 is**

above Disk 2 and Disk 2 is above Disk 3

So before Disk 3 can move, the two smaller disks must temporarily leave Peg A. Where do they go? **To Peg B.**

But now another problem appears. **How do we move Disk 2 to Peg B if Disk 1 is still sitting on top of it?** Again, space must be created. So Disk 1 must first move somewhere else.

Notice what just happened.

The original problem: **Move 3 disks** → Quietly became: **Move 2 disks** → And then became: **Move 1 disk**. This is the true recursive breakdown. Each larger problem continuously reduces itself into a smaller version of the same exact problem.

The Recursive Chain

This reduction continues repeatedly:

$$n \rightarrow (n - 1) \rightarrow (n - 2) \rightarrow (n - 3) \rightarrow \dots \rightarrow 1$$

Until we finally reach the base case: $n = 1$. At that point, no further decomposition is necessary. The move becomes direct.

And then something fascinating happens. The recursive calls begin returning one by one. This process is often called **unwinding**. The smaller solved problems now help solve the larger unfinished ones waiting above them.

So Recursion is not only about "going deeper". It is also about returning upward after the smaller work is complete.

Visualizing the Strategy with Three Disks

Let us follow the complete strategic flow again.

We begin with: Disk 1; Disk 2; and Disk 3. All stacked on Peg A.

→ Our ultimate objective is: Move Disk 3 to Peg C.

- But Disk 3 is blocked. So first: Move Disk 1 and Disk 2 to Peg B
- But Disk 2 is blocked by Disk 1. Therefore:
 - Move Disk 1 to Peg C

- Move Disk 2 to Peg B
- Move Disk 1 to Peg B
- Now Peg A finally contains only Disk 3. So: Move Disk 3 to Peg C
- Half the puzzle is complete. Now we must rebuild the smaller stack on top of Disk 3. So:
 - Move Disk 1 to Peg A
 - Move Disk 2 to Peg C
 - Move Disk 1 to Peg C
- And the puzzle is solved.

The beautiful part is this, the exact same strategy works for: 3 disks, 10 disks, 50 disks, even 100 disks. Only the recursive depth changes. The philosophy remains identical.

Why the Recursive Solution Feels Elegant

The Tower of Hanoi is often considered one of the greatest demonstrations of Recursion because it captures the essence of recursive thinking perfectly.

The algorithm does not try to solve everything immediately. Instead, it reduces the problem, solves a smaller version, returns to continue the larger version, repeats until completion

The recursive calls continuously shrink the problem size: $n \rightarrow n - 1 \rightarrow n - 2 \rightarrow \dots \rightarrow 1$

And after reaching the base case, the solution unfolds backward naturally.

This is why the Tower of Hanoi feels elegant. The complexity of the overall puzzle emerges from a very small recursive rule repeated consistently.

And that is one of the deepest lessons in Computer Science:

Large problems are often just smaller problems repeated carefully.

Summary

The Tower of Hanoi may initially look like a simple puzzle involving disks and pegs. But underneath it lies one of the purest demonstrations of Recursion ever

created.

The problem teaches us how Recursion truly works:

- Reduce a large problem into smaller versions of itself
- Continue reducing until reaching the base case
- Return step-by-step to solve the original problem

Instead of solving everything directly, the recursive strategy focuses on creating space for larger actions by temporarily moving smaller components away first.

And that idea appears far beyond the Tower of Hanoi itself. You will eventually encounter the same recursive philosophy in various other algorithms: Divide and Conquer; Dynamic Programming; Backtracking problems; Searching techniques; Artificial Intelligence, etc.

The Tower of Hanoi therefore becomes more than a puzzle. It becomes a model of recursive thinking itself. A problem where elegance emerges not from brute force, but from decomposition, reduction, and trust in smaller solutions.

And perhaps that is why the Tower of Hanoi remains timeless in Computer Science education. Because once you understand *why* the recursive strategy works here, you begin to understand the true beauty of Recursion itself.

You can also try the Tower of Hanoi yourself here:

<https://www.mathsisfun.com/games/towerofhanoi.html>

References

<https://www.geeksforgeeks.org/dsa/c-program-for-tower-of-hanoi/>

<https://www.youtube.com/watch?v=rf6uf3jNjbo>